

Herança, composição e polimorfismo

Quando uma classe deve herdar de outra, quando é melhor ela ter a outra dentro de si, e como um mesmo verbo ganha várias implementações.

O que veremos hoje

BLOCO 1

Herança

Uma classe que é um caso especial de outra: `super()`, sobrescrita e o teste do "é um".

BLOCO 2

Composição

Uma classe que tem a outra dentro de si, e o caso em que herdar quebra a regra que protegemos na Aula 6.

BLOCO 3

Polimorfismo

Um verbo, várias implementações: descontos trocáveis na Comanda, sem nenhum `if`.

De onde viemos: a Comanda virou classe

PONTO 1

Função virou método

O `calcular_total(itens)` da Aula 4 virou o método `total` da classe `Comanda`, que já enxerga os próprios itens pelo `self`.

PONTO 2

Invariante

Uma regra que vale sempre. Na `Comanda`: nenhum item entra com preço negativo.

PONTO 3

Encapsulamento

O underscore em `_itens` avisa "não mexa direto"; a proteção de verdade mora em `adicionar_item`, que valida antes de aceitar.

A Aula 6 respondeu como **uma** classe protege seus dados. Hoje a pergunta é como **várias** classes se relacionam sem quebrar essa proteção.

A Comanda, como ela terminou em 22/09

```
DOMINIO/COMANDA.PY · AULA 6

1 class Comanda:
2     def __init__(self, mesa):
3         self.mesa = mesa
4         self._itens = []
5
6     @property
7     def total(self):
8         return sum(preco for _, preco in self._itens)
9
10    def adicionar_item(self, nome, preco):
11        if preco < 0:
12            raise ValueError("preço não pode ser negativo")
13        self._itens.append((nome, preco))
```

CONFIRA Cada item era uma tupla (nome, preco). A validação em `adicionar_item` garantia a invariante: preço nunca negativo. Hoje partimos exatamente daqui.

E o exercício que fizemos com ela

EXERCÍCIO DA AULA 6 · RESOLVIDO

```
1 c1 = Comanda(mesa=5)
2 c1.adicionar_item("água", 6)
3 c1.adicionar_item("prato", 32)
4 c1.adicionar_item("sobremesa", 14)
5 print(c1.total) # 52
```

CONFIRA Guarde esses três itens e o total 52: vamos reaproveitá-los no Bloco 3, quando a Comanda ganhar desconto.

Quatro perguntas sobre o mesmo domínio

O domínio da equipe responde a uma pergunta diferente a cada aula.

AULA 3 · RESPONDIDO

Como guardar o dado?

Variáveis, tipos e coleções: listas, tuplas e dicionários.

AULA 4 · RESPONDIDO

Como organizar o comportamento?

Funções com retorno, módulos e a primeira suíte pytest.

AULA 6 · RESPONDIDO

Como juntar dado e comportamento?

A classe, com invariante protegida no ponto de entrada.

HOJE · EM ABERTO

Como as classes se relacionam?

Herdar, compor ou trocar de comportamento, sem perder a proteção.

Um sistema real nunca tem uma classe só. A forma como elas se ligam decide se o código aguenta mudança.

Primeiro cada classe se protege. Depois, elas aprendem a trabalhar juntas.

O restaurante agora vende pratos e bebidas. A bebida tem volume em ml; o prato, tempo de preparo. Vocês criariam duas classes independentes, uma classe com vários ifs, ou outro caminho?

As três respostas funcionam hoje. A diferença aparece no dia em que chegar o terceiro tipo de item. É dessa diferença que a aula trata.

BLOCO 1

Herança

Quando uma classe é um caso especial de outra, ela pode herdar tudo o que a outra já sabe fazer e acrescentar só o que é dela.

O que é herança

Herança é criar uma classe nova (a subclasse) a partir de uma existente (a superclasse). A subclasse recebe todos os atributos e métodos da superclasse e pode acrescentar ou substituir os seus.

NA LITERATURA

Python Software Foundation

O tutorial oficial apresenta a herança com a sintaxe class Derivada(Base): e explica que, se um atributo não é achado na subclasse, a busca continua na classe base.

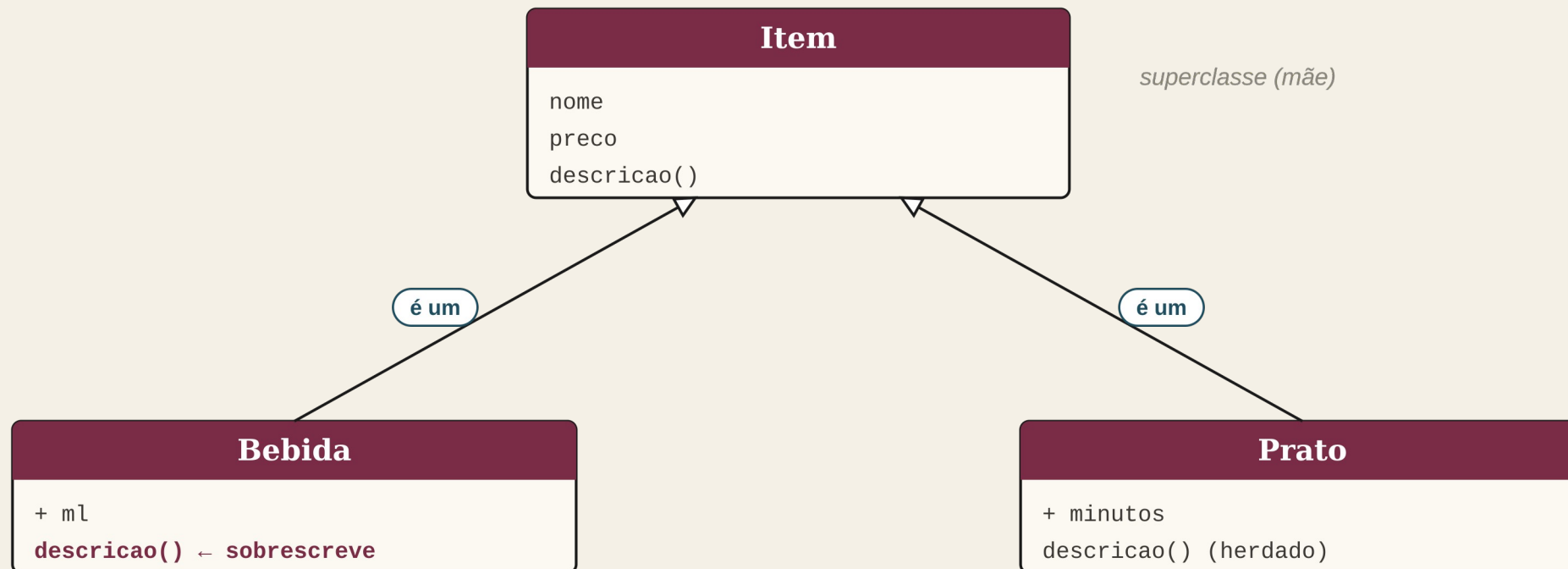
NA PRÁTICA

O teste do "é um"

Bebida é um Item? Sim. Prato é um Item? Sim. Se a frase soa natural, a herança é candidata. Se soa estranha, não é.

PYTHON SOFTWARE FOUNDATION. The Python Tutorial, seção 9.5: Inheritance. docs.python.org.

A árvore de herança do cardápio



A seta de ponta vazada aponta para a mãe. O que está na mãe, as filhas já têm; o "+" é só o que cada filha acrescenta.

Item é a superclasse; Bebida e Prato são subclasses. O código de Item é escrito uma vez e vale para as duas.

Primeiro passo: o item deixa de ser tupla

```
DOMINIO/ITEM.PY

1 class Item:
2     def __init__(self, nome, preco):
3         if preco < 0:
4             raise ValueError("preço não pode ser negativo")
5         self.nome = nome
6         self.preco = preco
7
8     def descricao(self):
9         return f"{self.nome}: R$ {self.preco}"
```

CONFIRA A tupla ("água", 6) da Aula 6 vira Item("água", 6). E a invariante muda de casa: agora é o próprio item que se recusa a nascer com preço negativo.

Bebida é um Item: herança e super()

```
DOMINIO/ITEM.PY

1 class Bebida(Item):
2     def __init__(self, nome, preco, ml):
3         super().__init__(nome, preco)
4         self.ml = ml
5
6 suco = Bebida("suco", 8, 300)
7 print(suco.nome, suco.preco, suco.ml) # suco 8 300
8 Bebida("erro", -1, 300) # ValueError
```

CONFIRA `super().__init__` chama o `__init__` de `Item`: nome, preço e a validação vêm de lá, sem copiar uma linha. `Bebida` só acrescenta o `ml`.

super(): chamar a versão da classe mãe

super() devolve um objeto que encaminha a chamada para a superclasse. Dentro de `__init__`, `super().__init__(...)` garante que a parte herdada do objeto seja preparada do jeito que a classe mãe definiu, inclusive com as validações dela.

O ERRO COMUM

Esquecer o super()

Se Bebida define `__init__` e não chama `super().__init__`, o nome e o preço nunca são gravados, e a validação de preço nunca roda.

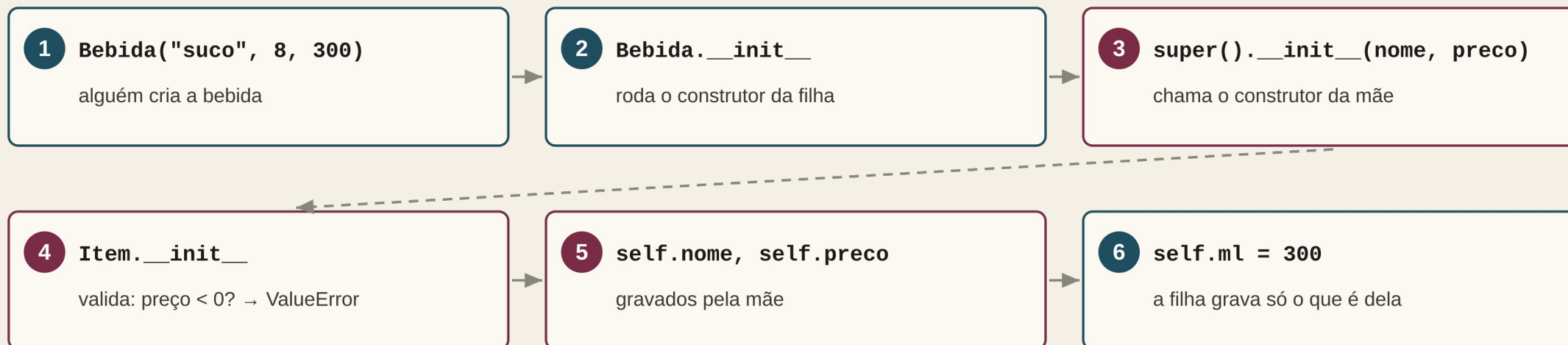
NA LITERATURA

Luciano Ramalho, Python Fluente

Ramalho recomenda usar sempre `super()` em vez de chamar a classe mãe pelo nome, porque ele respeita a ordem de busca de métodos do Python.

RAMALHO, L. Python Fluente. 2. ed. Novatec, 2022, capítulo "Herança: para o bem ou para o mal". PYTHON SOFTWARE FOUNDATION. Built-in Functions: `super()`.

O que acontece quando uma Bebida nasce



■ código da Bebida (filha)

■ código do Item (mãe), reaproveitado via `super()`

Sem o passo 3, os passos 4 e 5 nunca acontecem: a bebida nasce sem nome, sem preço e sem validação.

Seis passos, dois donos: a filha começa e termina, mas no meio entrega o trabalho comum para a mãe.

Sobrescrita: a subclasse muda um comportamento

```
DOMINIO/ITEM.PY

1 class Bebida(Item):
2     def __init__(self, nome, preco, ml):
3         super().__init__(nome, preco)
4         self.ml = ml
5
6     def descricao(self):
7         base = super().descricao()
8         return f"{base} ({self.ml} ml)"
```

CONFIRA Bebida redefine descricao (sobrescrita), mas reaproveita a versão de Item com super(). Resultado: "suco: R\$ 8 (300 ml)".

PAUSA PARA PENSAR

Como Bebida e Prato em relação a Item: no domínio da equipe, existem duas classes que repetem os mesmos atributos e passam fácil no teste do "é um"?

Se existirem, são candidatas a uma superclasse comum. Se a frase "X é um Y" soar forçada, anote: provavelmente o caso é do Bloco 2.

O teste mais rigoroso: substituição

Uma subclasse só é uma boa subclasse se puder ser usada em qualquer lugar onde a superclasse é esperada, sem que o programa perceba a troca. É o Princípio da Substituição de Liskov.

NA LITERATURA

Barbara Liskov e Jeannette Wing

Em 1994, as autoras definiram subtipo pelo comportamento: tudo o que se pode garantir sobre o tipo base precisa continuar valendo para o subtipo.

NA COMANDA

Bebida passa no teste

Onde a Comanda espera um Item, ela aceita uma Bebida: tem nome, tem preço válido e tem descricao(). Nada quebra.

LISKOV, B.; WING, J. A behavioral notion of subtyping. ACM Transactions on Programming Languages and Systems, v. 16, n. 6, 1994.

Qual é a saída deste código?

EXERCÍCIO

```
1 class Prato(Item):
2     def __init__(self, nome, preco, minutos):
3         super().__init__(nome, preco)
4         self.minutos = minutos
5
6 p = Prato("feijoadada", 32, 25)
7 print(p.descricao())
8 print(isinstance(p, Item))
```

"feijoada: R\$ 32", depois True



EXERCÍCIO · RESOLVIDO

```
1 p = Prato("feijoada", 32, 25)
2 print(p.descricao()) # feijoada: R$ 32
3 print(isinstance(p, Item)) # True
```

CONFIRA Prato não sobrescreveu descricao, então usa a de Item, sem os minutos. E um Prato é um Item: isinstance confirma.

Onde o Python procura um método



O Python procura o método primeiro na classe do objeto e, se não achar, sobe para a mãe.

Por isso a Bebida mostra os ml e o Prato mostra só nome e preço, como no exercício.

A busca sobe da filha para a mãe. Sobrescrever é só colocar o método na filha: ela é achada primeiro.

O que sustenta uma boa herança

01 É um caso especial

A frase "subclasse é uma superclasse" precisa soar natural.

02 `super()` preserva a mãe

A parte herdada nasce do jeito da superclasse, com as validações dela.

03 Substituível sem susto

Onde a mãe é esperada, a filha funciona. É o teste de Liskov.

Herança é poderosa, e por isso perigosa. O próximo bloco mostra quando ela quebra o que a Aula 6 protegeu.

BLOCO 2

Composição

Em vez de ser um caso especial de outra classe, a classe tem a outra dentro de si e usa o que ela sabe fazer.

O que é composição

Composição é quando um objeto guarda outros objetos como atributos e delega a eles parte do trabalho. A relação é "tem um": a Comanda tem itens; ela não é um item.

NA LITERATURA

Gamma, Helm, Johnson e Vlissides

O livro de padrões de projeto de 1994, conhecido como GoF, resume a lição em uma frase: prefira composição de objetos a herança de classes.

VOCÊS JÁ FIZERAM

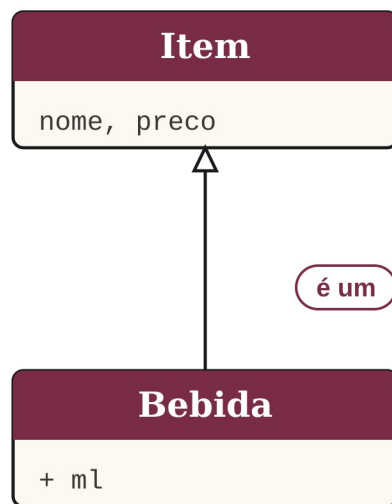
A Comanda da Aula 6

`self._itens` é uma lista guardada dentro da Comanda. Isso já é composição: a Comanda tem uma lista e usa o que a lista sabe fazer.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

Duas relações, dois desenhos

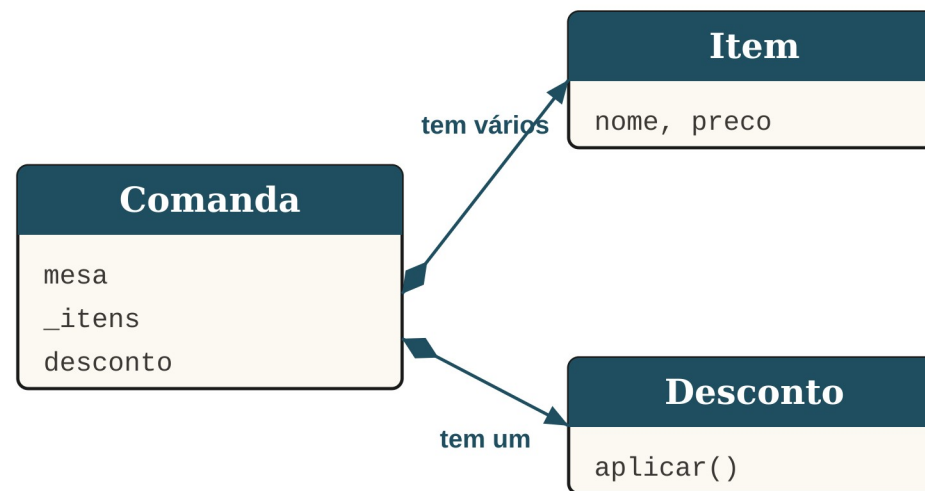
HERANÇA · “é um”



é um

`class Bebida(Item)`

COMPOSIÇÃO · “tem um”



`self._itens = [ ] · self.desconto = ...`

Teste rápido: leia a frase em voz alta. Se “X é um Y” soar estranho, a relação é “tem um”.

Seta vazada: herança (é um). Losango cheio: composição (tem um). Os mesmos símbolos da UML, usados em qualquer livro de orientação a objetos.

"É um" ou "tem um"?

Frases	Relação	Solução
Bebida é um Item	É um	Herança: <code>class Bebida(Item)</code>
Comanda tem itens	Tem um	Composição: <code>self._itens = []</code>
Comanda é uma lista	Soa estranho	Não herde: é o caso do próximo slide
Pedido tem um cliente	Tem um	Composição: <code>self.cliente = cliente</code>

A tentação: Comanda herda de list

DOMINIO/COMANDA_ERRADA.PY

```
1 class Comanda(list):
2     def append(self, item):
3         if item.preco < 0:
4             raise ValueError("preço não pode ser negativo")
5         super().append(item)
6
7     @property
8     def total(self):
9         return sum(i.preco for i in self)
```

CONFIRA Parece esperto: a Comanda ganha de graça len(), for, índice, ordenação... Mas ela também ganha tudo o que uma lista sabe fazer, inclusive o que não devia.

O que o último print mostra?

EXERCÍCIO

```
1 class Falso: # objeto qualquer, sem validação
2     def __init__(self, preco):
3         self.preco = preco
4
5 c = Comanda() # a versão que herda de list
6 c.append(Item("água", 6))
7 c.extend([Falso(-100)])
8 c.insert(0, Falso(-50))
9 print(c.total)
```

-144: a invariante quebrou



EXERCÍCIO · RESOLVIDO

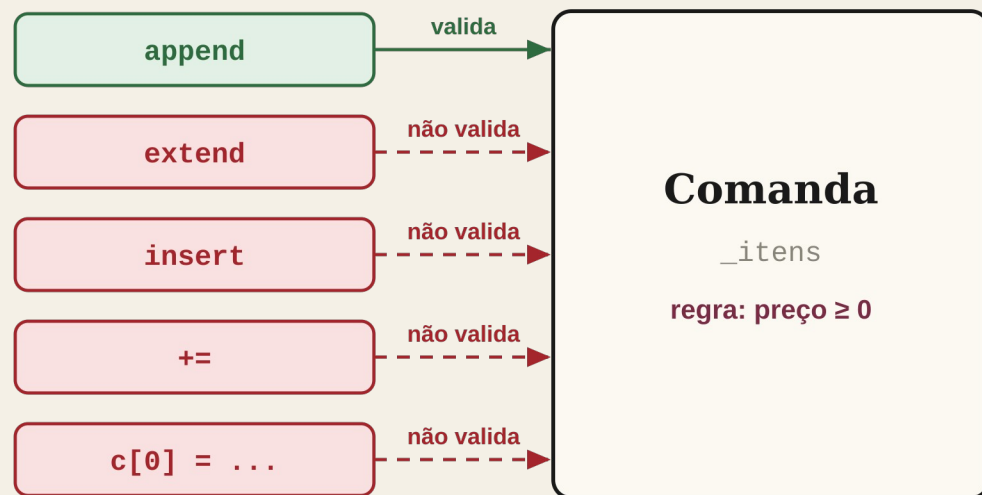
```
1 c.append(Item("água", 6))    # validado: entra
2 c.extend([Falso(-100)])      # NÃO passa por append
3 c.insert(0, Falso(-50))      # NÃO passa por append
4 print(c.total)               # 6 - 100 - 50 = -144
```

CONFIRA

Ao herdar de list, a Comanda herdou extend, insert e outras portas de entrada que ignoram a validação. A regra da Aula 6 ("preço nunca negativo") caiu sem nenhum erro.

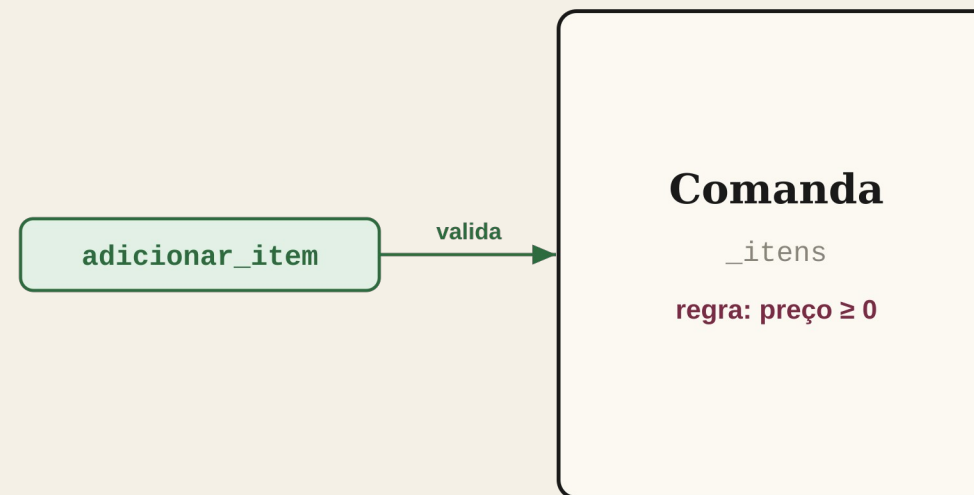
Por que a herança furou a regra

`class Comanda(list) x`



cinco portas, só uma com vigia: total -144

`Comanda com self._itens ✓`



uma porta, com vigia: regra garantida

Herdando de list, a Comanda ganhou portas que não passam pela validação. Compondo, ela decide quais portas existem.

Por que a herança quebrou a proteção

Quem herda recebe a interface inteira da classe mãe, e não só a parte que queria. Cada método herdado é uma porta de entrada que a subclasse precisa vigiar, e é fácil esquecer uma. Com composição, a Comanda expõe só as portas que ela escolheu abrir.

NA LITERATURA

Luciano Ramalho, Python Fluente

Ramalho alerta que herdar diretamente de tipos embutidos como list e dict é propenso a erro, porque os métodos embutidos em geral não chamam os métodos sobrescritos pela subclasse.

O NOME DO PROBLEMA

Acoplamento à classe mãe

A subclasse depende de detalhes internos da mãe. Uma mudança lá em cima pode quebrar as filhas sem que ninguém perceba.

RAMALHO, L. Python Fluente. 2. ed. Novatec, 2022, capítulo "Herança: para o bem ou para o mal", seção sobre herdar de tipos embutidos.

A versão certa: a Comanda tem uma lista

```
DOMINIO/COMANDA.PY

1 class Comanda:
2     def __init__(self, mesa):
3         self.mesa = mesa
4         self._itens = []          # composição: tem uma lista
5
6     def adicionar_item(self, item):
7         self._itens.append(item)  # única porta de entrada
8
9     @property
10    def total(self):
11        return sum(i.preco for i in self._itens)
```

CONFIRA A Comanda da Aula 6, agora com objetos Item. Sem extend nem insert: só adicionar_item. E o Item já nasce validado.

PAUSA PARA PENSAR

No domínio da equipe, alguma classe herda de list, de dict ou de outra classe só para "aproveitar os métodos"? A frase "X é um Y" soa natural para ela?

Se não soa, troquem a herança por um atributo guardado dentro da classe. É a mudança mais comum na entrega de hoje.

Quando a composição vence

- | | | |
|-----------|---|---|
| 01 | A relação é "tem um" | Comanda tem itens, pedido tem cliente, empréstimo tem livro. |
| <hr/> | | |
| 02 | A invariante precisa de uma porta só | Compondo, a classe escolhe quais portas abrir. Herdando, recebe todas. |
| <hr/> | | |
| 03 | O comportamento pode mudar | Um objeto guardado dentro de outro pode ser trocado por outro. É o próximo bloco. |

E se o objeto guardado dentro da Comanda puder ser trocado por outro que faz a mesma coisa de outro jeito?

BLOCO 3

Polimorfismo

Objetos diferentes respondem ao mesmo verbo, cada um do seu jeito, e quem chama não precisa saber qual é qual.

O que é polimorfismo

Polimorfismo é a capacidade de objetos de tipos diferentes responderem à mesma mensagem, ao mesmo nome de método, cada um com a sua implementação. Quem chama só precisa saber o verbo.

VOCÊS JÁ USAM

len() e o operador +

`len("abc")`, `len([1, 2])` e `len({"a": 1})` funcionam com três tipos diferentes. `2 + 3` soma números; `"a" + "b"` junta textos. O mesmo verbo, várias implementações.

NA LITERATURA

Python Software Foundation

O glossário do Python chama de duck typing: o código não pergunta o tipo do objeto, só se ele tem o método necessário. "Se anda como pato e grasna como pato..."

PYTHON SOFTWARE FOUNDATION. Glossary: duck-typing. docs.python.org.

Três descontos, um verbo: aplicar

```
DOMINIO/DESCONTO.PY

1 class SemDesconto:
2     def aplicar(self, subtotal):
3         return subtotal
4 class Percentual:
5     def __init__(self, pct):
6         self.pct = pct
7     def aplicar(self, subtotal):
8         return subtotal * (100 - self.pct) / 100
9 class ValorFixo:
10    def __init__(self, valor):
11        self.valor = valor
12    def aplicar(self, subtotal):
13        return max(subtotal - self.valor, 0)
```

CONFIRA Nenhuma herança entre elas. O que as une é o método aplicar(subtotal): mesmo nome, mesmo contrato.

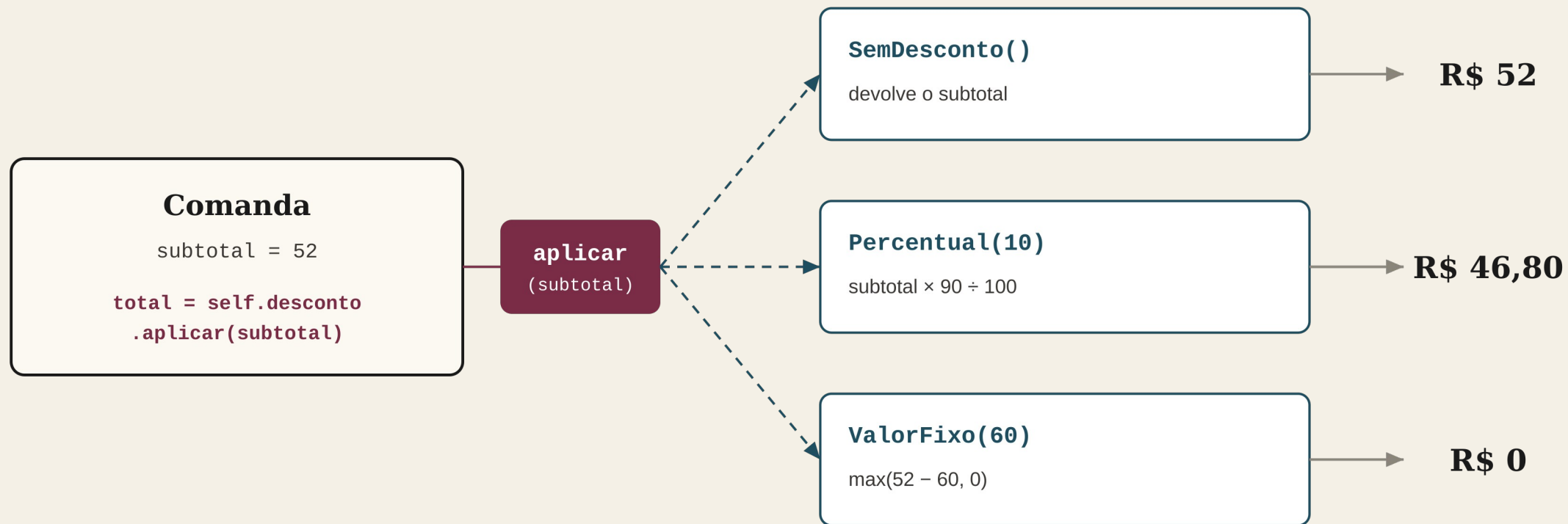
Composição e polimorfismo juntos

```
DOMINIO/COMANDA.PY

1 class Comanda:
2     def __init__(self, mesa, desconto=None):
3         self.mesa = mesa
4         self._itens = []
5         self.desconto = desconto or SemDesconto()
6
7     @property
8     def subtotal(self):
9         return sum(i.preco for i in self._itens)
10
11    @property
12    def total(self):
13        return self.desconto.aplicar(self.subtotal)
```

CONFIRA A Comanda tem um desconto (composição) e só chama aplicar (polimorfismo). Não sabe qual desconto é. Nenhum if.

Uma tomada, três aparelhos



Como uma tomada: a Comanda só conhece o formato do plugue (aplicar). Qualquer desconto com esse plugue encaixa.

A Comanda conhece só o formato do plugue: um método aplicar que recebe o subtotal e devolve o total.

Sem polimorfismo, com polimorfismo

SEM · CADEIA DE IFS

if tipo == "percentual": ...

Cada desconto novo obriga a abrir a Comanda e acrescentar mais um elif. A classe cresce e quebra a cada regra de marketing.

COM · OBJETOS TROCÁVEIS

self.desconto.aplicar(...)

Desconto novo é uma classe nova com aplicar. A Comanda não muda uma linha, e os testes dela continuam verdes.

Qual é o total de cada comanda?

EXERCÍCIO

```
1  itens = [Item("água", 6), Item("prato", 32), Item("sobremesa", 14)]
2
3  for desc in [SemDesconto(), Percentual(10), ValorFixo(60)]:
4      c = Comanda(mesa=5, desconto=desc)
5      for i in itens:
6          c.adicionar_item(i)
7      print(type(desc).__name__, c.total)
```

52, depois 46.8, depois 0

EXERCÍCIO · RESOLVIDO

```
1 SemDesconto 52      # 52
2 Percentual 46.8      # 52 x 90 / 100
3 ValorFixo 0          # max(52 - 60, 0)
```

CONFIRA O mesmo laço, a mesma Comanda, o mesmo verbo aplicar, três resultados. E o `max(..., 0)` do ValorFixo protege outra invariante: total nunca negativo.

Um teste para todos os descontos

```
TESTS/TEST_COMANDA.PY

1 import pytest
2
3 @pytest.mark.parametrize("desconto, esperado", [
4     (SemDesconto(), 52),
5     (Percentual(10), 46.8),
6     (ValorFixo(60), 0),
7 ])
8 def test_total_com_desconto(desconto, esperado):
9     c = Comanda(mesa=5, desconto=desconto)
10    for nome, preco in [("água", 6), ("prato", 32), ("sobremesa", 14)]:
11        c.adicionar_item(Item(nome, preco))
12    assert c.total == pytest.approx(esperado)
```

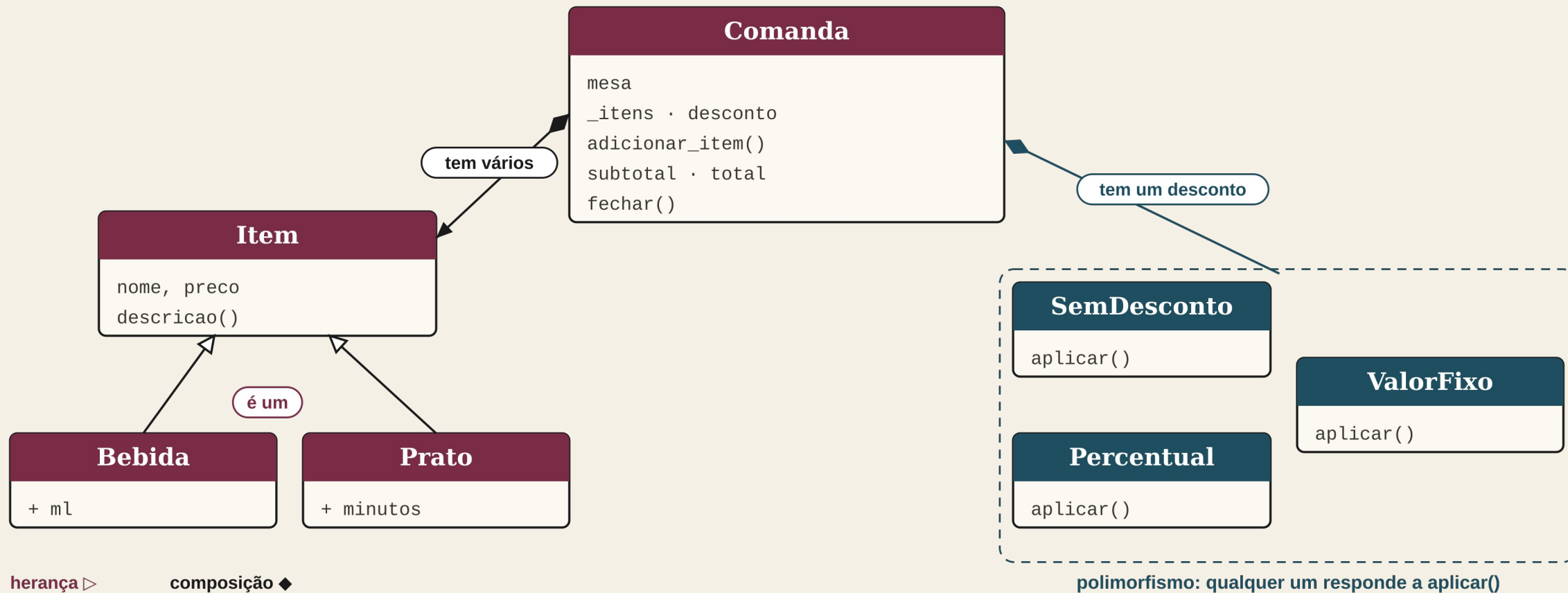
CONFIRA `pytest.mark.parametrize` roda o mesmo teste uma vez para cada desconto. `pytest.approx` evita falso erro com número decimal, lembrando a Aula 3: ponto flutuante não é exato.

O que o polimorfismo resolve

- 01 Um verbo, várias formas** Objetos diferentes respondem ao mesmo método, cada um do seu jeito.
- 02 Quem chama não pergunta o tipo** É o duck typing do Python: basta ter o método.
- 03 Regra nova, classe nova** Acrescentar comportamento não obriga a mexer no que já funciona.

Na Aula 8 esse "contrato" ganha forma explícita, com classes abstratas, e é comparado ao Java.

O núcleo completo, num desenho só



Tudo o que a equipe entrega hoje cabe num diagrama assim: classes, quem é um, quem tem um, e quem responde ao mesmo verbo.

O que fica de hoje

Ponto	Regra prática
Herança	Só quando a frase "é um" soa natural e a subclasse substitui a mãe sem susto.
super()	A parte herdada nasce do jeito da mãe, com as validações dela.
Composição	Quando a relação é "tem um", ou quando a invariante precisa de uma porta só.
Polimorfismo	Mesmo verbo, várias implementações. Regra nova vira classe nova, não if novo.

Prefira compor. Herde quando for, de fato, um caso especial.

Núcleo orientado a objetos com testes

- | | | |
|----|---|--|
| 01 | Classes do domínio | As 3 a 5 classes centrais do projeto, cada uma com responsabilidade clara, no pacote de domínio. |
| 02 | Invariantes protegidas | Pelo menos duas regras de negócio validadas no ponto de entrada, cada uma com um teste que espera o ValueError. |
| 03 | Relações justificadas | Uma composição, e herança só se passar no teste do "é um". Um comentário de uma linha justificando cada escolha. |
| 04 | Um polimorfismo | Um verbo com pelo menos duas implementações trocáveis, como os descontos da Comanda. |
| 05 | Suíte verde e repositório em dia | pytest passando, README com como rodar os testes, e o último commit enviado ao GitHub até o fim da aula. |

Vale 50% do primeiro processo. O que é avaliado é a qualidade das decisões, não a quantidade de classes.

Lista de exercícios

- | | | |
|----|------------------------------------|--|
| 01 | O teste do "é um" | Listem todas as heranças do projeto e escrevam a frase "X é um Y" para cada uma. A que soar estranha vira composição. |
| 02 | Uma subclasse com super() | Criem ou revisem uma subclasse que chame super().__init__ e provem, com um teste, que a validação da mãe continua valendo. |
| 03 | Sobrescrita que reaproveita | Sobrescrevam um método na subclasse chamando a versão da mãe com super(), como a descrição da Bebida. |
| 04 | Quebrem de propósito | Se alguma classe herda de list ou dict, achem um método herdado que fura a invariante. Depois, troquem por composição. |
| 05 | Um verbo, duas classes | Criem duas classes com o mesmo método (ex.: calcular_multa, calcular_frete) e usem as duas num mesmo laço, sem if de tipo. |
| 06 | parametrize | Escrevam um teste com pytest.mark.parametrize que rode as implementações do exercício 5. |

Tudo no domínio que vocês já têm. Não é projeto novo: é o mesmo núcleo, com relações mais bem escolhidas.

REFERÊNCIAS

De onde veio cada afirmação desta aula

HERANÇA E SUPER()

PYTHON SOFTWARE FOUNDATION. The Python Tutorial, 9.5 Inheritance; Built-in Functions: `super()`. docs.python.org.

`sintaxe, busca de atributos e super()`

RAMALHO, L. Python Fluente. 2. ed. Novatec, 2022.

"Herança: para o bem ou para o mal"; herdar de tipos embutidos

LISKOV, B.; WING, J. A behavioral notion of subtyping. ACM TOPLAS, v. 16, n. 6, 1994.

`princípio da substituição`

COMPOSIÇÃO E POLIMORFISMO

GAMMA, E. et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

`prefira composição a herança`

PYTHON SOFTWARE FOUNDATION. Glossary: duck-typing. docs.python.org.

`polimorfismo sem herança`

PYTEST. How to parametrize fixtures and test functions. docs.pytest.org.

`pytest.mark.parametrize e approx`
